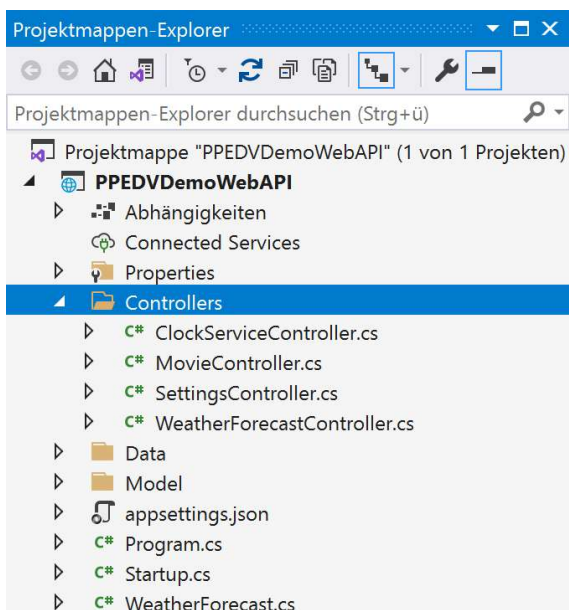


Controller Klassen



Controller-Verzeichnis



Rechtsklick auf Controller-Verzeichnis

-> Add

-> New Controller

2

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Controller registrieren (IOC)

Durch die Verwendung von `AddControllers`, muss beim `UseEndpoints` der `MapController` verwendet werden.

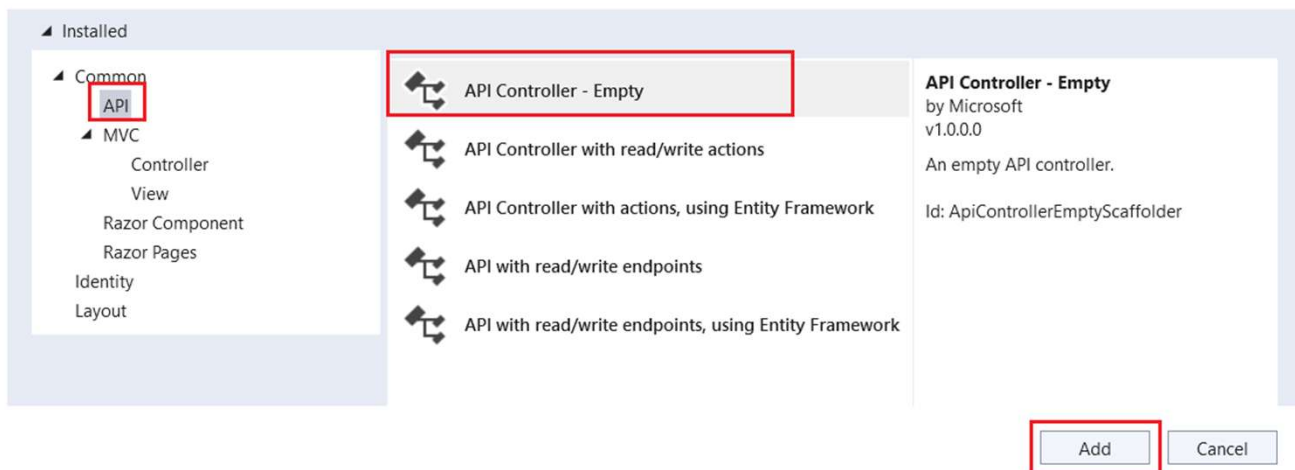
Dieser stellt die MVC Logik und leitet Request an die jeweilige Action-Methode weiter, stellt die Navigation, Binding, Filtering und viele weitere MVC Feature bereit

Endpoint `MapControllers`, leitet die Request an die jeweiligen Controller und ActionMethoden weiter.

```
// Add services to the container.  
builder.Services.AddControllers();  
  
// Learn more about configuring Swagger/OpenAPI  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();  
  
app.MapControllers();
```

WebAPI – Controller Template

Add New Scaffolded Item



4

IN-COURSE REMARKS

g
g

PREPARATION REMARKS

g
g

ControllerBase - Klasse

- Eine Web-API besteht aus mindestens einer Controllerklasse, die von ControllerBase abgeleitet ist.
- bietet viele Eigenschaften und Methoden, die für die Verarbeitung von HTTP-Anforderungen hilfreich sind.

```
[ApiController]  
[Route(template: "[controller]")]  
3 Verweise  
public class WeatherForecastController : ControllerBase
```

- Verwenden Sie bei einer Web-API-Controller nicht die Basisklasse *Controller*:
 - Controller wird von ControllerBase abgeleitet und kann zusätzlich HTML-Inhalte (Views und mehr) ausgeben.
 - Ausnahme: Wenn Sie denselben Controller sowohl für Ansichten als auch für Web-APIs verwenden möchten, leiten Sie ihn von Controller ab.

5

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

ApiController-Attribut

Das [ApiController]-Attribut kann auf eine Controllerklasse angewendet werden, um folgende API-spezifische Verhalten mit einigen vordefinierten Konfigurationen zu aktivieren:

- Anforderung für das Attributrouting
- Automatische HTTP 400-Antworten
- Binding source parameter inference
- Ableiten der Multipart/form-data-Anforderung
- ProblemDetails für Fehler Status-Codes

6

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Attribut für mehrere Controller

Eine Möglichkeit, das Attribut für mehr als einen Controller zu verwenden, besteht darin, eine benutzerdefinierte Basiscontrollerklasse zu erstellen, die mit dem [ApiController]-Attribut definiert ist.

```
[ApiController]
```

1 Verweis

```
public class MyControllerBase : ControllerBase  
{  
    //...  
}
```

```
[Produces(MediaTypeNames.Application.Json)]
```

```
[Route(template: "[controller]")]
```

0 Verweise

```
public class PetsController : MyControllerBase  
{  
    //...  
}
```

7

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Automatische 400-Antwort

Durch das `[ApiController]`-Attribut wird bei Modellvalidierungsfehlern automatisch eine HTTP 400-Antwort ausgelöst.

```
[Produces(MediaTypeNames.Application.Json)]
[Route(template: "[controller]")]
0 Verweise
public class PetsController : MyControllerBase
{
    0 Verweise
    public IActionResult Insert(Movie movie)
    {
        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }

        return Ok();
    }
}
```

8

ControllerBase – Bereitgestellte Methoden/Properties

Methoden	Beschreibung
BadRequest	Gibt Statuscode 400 zurück.
NotFound	Gibt Statuscode 404 zurück.
PhysicalFile	Gibt eine Datei zurück.
TryUpdateModelAsync	Ruft die Modellbindung auf.
TryValidateModel	Ruft die Modellvalidierung auf.

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

WebApi Controller – Dependency Unterstützung

```
public class MovieController : ControllerBase
{
    private readonly MovieDbContext _context;
    private readonly IConfiguration _configuration;

    0 Verweise
    public MovieController(MovieDbContext context, IConfiguration configuration)
    {
        _context = context;
        _configuration = configuration;
    }
}
```

10

IN-COURSE REMARKS

g
g

PREPARATION REMARKS

g
g

WebAPI-Controller vs. MVC-Controller

- WebAPI Controller ist der Basis Controller -> ControllerBase
- Ideal für RESTful-Services, deren Schwerpunkt auf die Rückgaben von Daten liegt

```
namespace Microsoft.AspNetCore.Mvc
    public abstract class ControllerBase
```

- MVC-Controller erbt von ControllerBase ab
- Abstraktionsebene für zusätzliche Funktionen zur Unterstützung der Erstellung von Websites wie das Zurückgeben von Views und Verarbeiten von Razor-Views

```
namespace Microsoft.AspNetCore.Mvc
    public abstract class Controller : ControllerBase, IActionFilter, IAsyncActionFilter, IDisposable
```

Best Practices

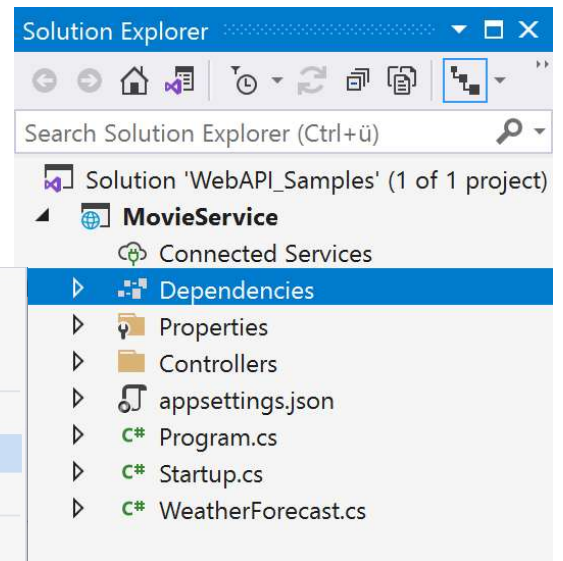
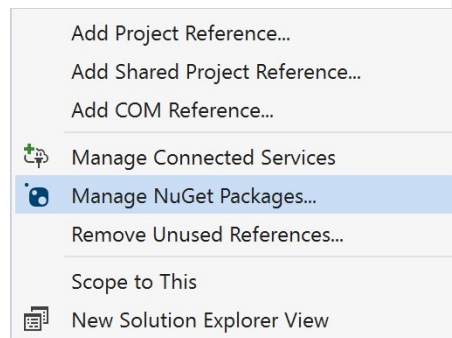
- **Dünne Controller:** Behalten Sie die Geschäftslogik aus Controllern heraus und in Diensten oder anderen Ebenen.
- **Verwenden Sie DTOs (Data Transfer Objects):** Vermeiden Sie die direkte Offenlegung Ihrer Datenbankmodelle; ordnen Sie sie DTOs zu.
- **Einheitliche Namenskonvention:** Befolgen Sie die RESTful-Benennungskonventionen für Aktionen.
- **Sichern Sie Ihre API:** Implementieren Sie Authentifizierung, Autorisierung und validieren Sie Eingaben, um Ihre API zu sichern.
- **Unit-Tests:** Schreiben Sie Unit-Tests für Ihre Controller, um Zuverlässigkeit und Wartbarkeit sicherzustellen.

RESTful-Walkthrough

How-To

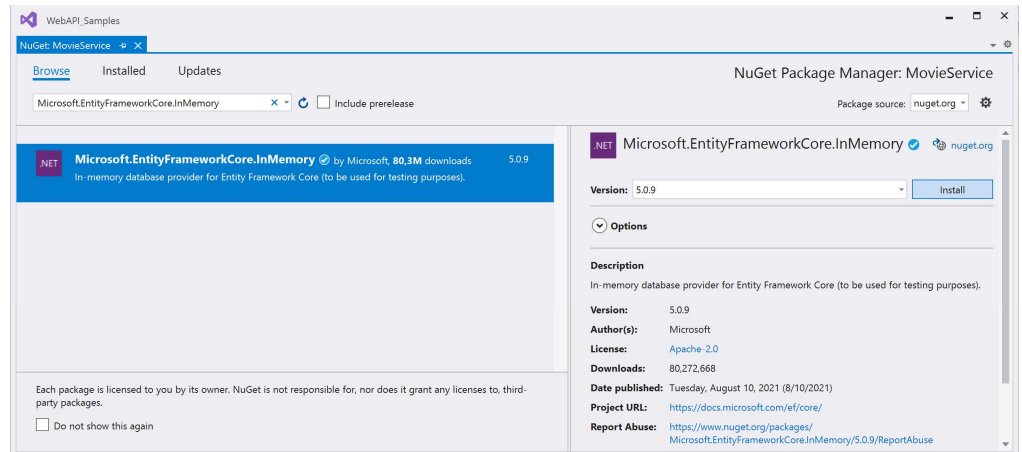
Erster RESTful-Service – Packages hinzufügen (Schritt 1)

- 1.) Nach dem Erstellen, sollte im Solution Explorer der „MovieService“ erscheinen.
- 2.) Rechtsklick auf die Projektmappe „MovieService“
- 3.) Im Popup-Menü -> „Manager NuGet Packages...“ auswählen



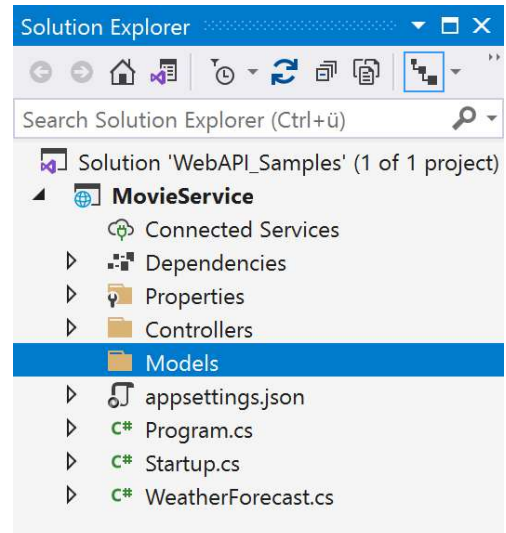
Erster RESTful-Service – Packages hinzufügen (Schritt 2)

- 1.) Klicke auf Tab „Browse“
- 2.) Suche nach dem Package: „Microsoft.EntityFrameworkCore.InMemory“
- 3.) Optional können Sie die Package-Version auswählen.
- 4.) Weiter mit „Install“



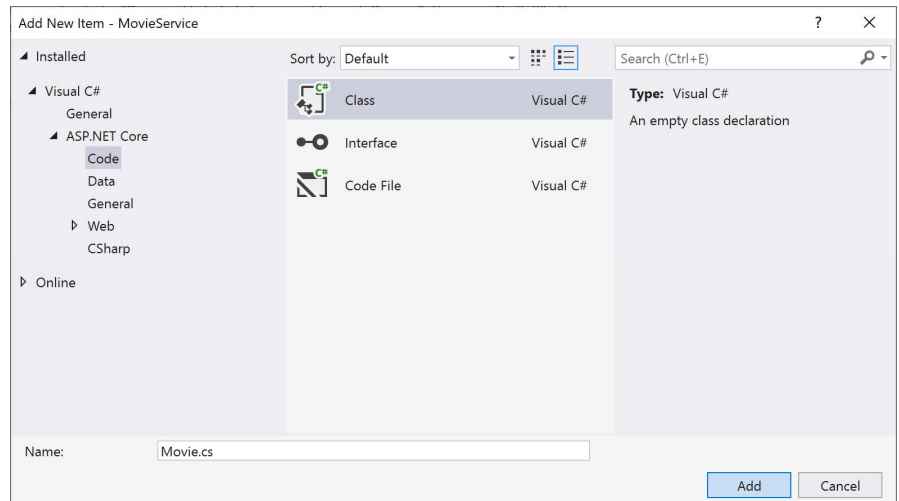
Erster RESTful-Service – Models Verzeichnis anlegen

- 1.) Rechtsklick auf Projektmappe „MovieService“
- 2.) Im Popup-Menü -> Add -> Folder
- 3.) Foldername ist „Models“



Erster RESTful-Service – Movie-Klasse hinzufügen

- 1.) Rechtsklick auf den Folder „Models“
- 2.) Im Popup-Menü -> Add > Add New Item auswählen
- 3.) Klassen - Template auswählen
- 4.) Klasse nennen wir „Movie.cs“
- 5.) Weiter mit „Add“

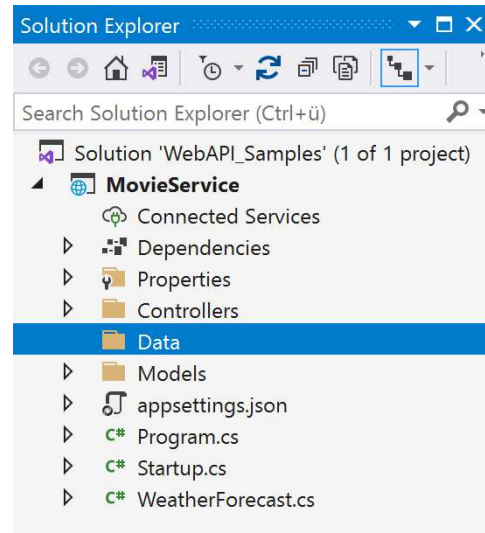


Erster RESTful-Service – Aufbau Movie.cs

```
namespace MovieService.Models
{
    0 references
    public class Movie
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Title { get; set; }
        0 references
        public string Description { get; set; }
        0 references
        public decimal Price { get; set; }
    }
}
```

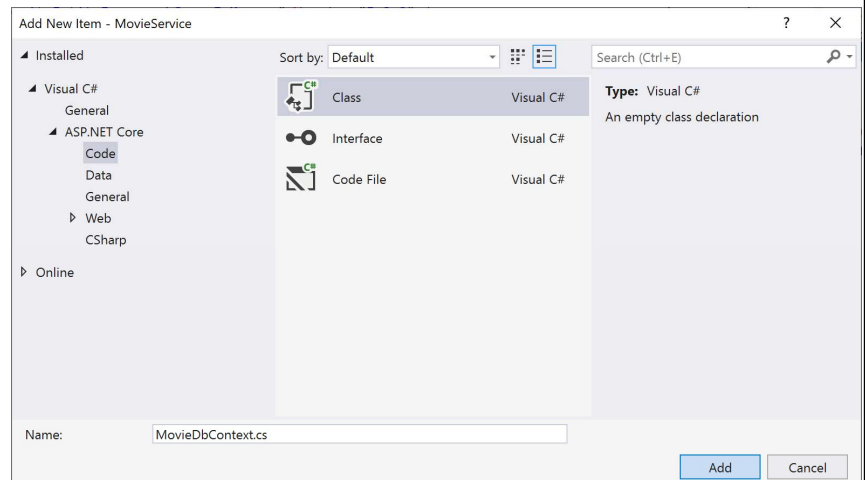
Erster RESTful-Service – Data Verzeichnis anlegen

- 1.) Rechtsklick auf Projektmappe „MovieService“
- 2.) Im Popup-Menü -> Add -> Folder
- 3.) Foldername ist „Data“



Erster RESTful-Service – MovieDbContext-Klasse hinzufügen

- 1.) Rechtsklick auf den Folder „Data“
- 2.) Im Popup-Menü -> Add > Add New Item auswählen
- 3.) Klassen - Template auswählen
- 4.) Klasse nennen wir „MovieDbContext.cs“
- 5.) Weiter mit „Add“



Erster RESTful-Service – Aufbau MovieDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using MovieService.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace MovieService.Data
{
    2 references
    public class MovieDbContext : DbContext
    {
        0 references
        public MovieDbContext(DbContextOptions<MovieDbContext> options)
            :base(options)
        {
        }
        0 references
        public DbSet<Movie> Movies { get; set; }
    }
}
```

Erster RESTful-Service – Registrieren von MovieDbContext

- 1) Öffne Startup.cs
- 2) Fügen Sie folgendes using hinzu -> `using Microsoft.EntityFrameworkCore;`
- 3) Erweitere die Methode Configure Services mit folgenden Codezeilen:

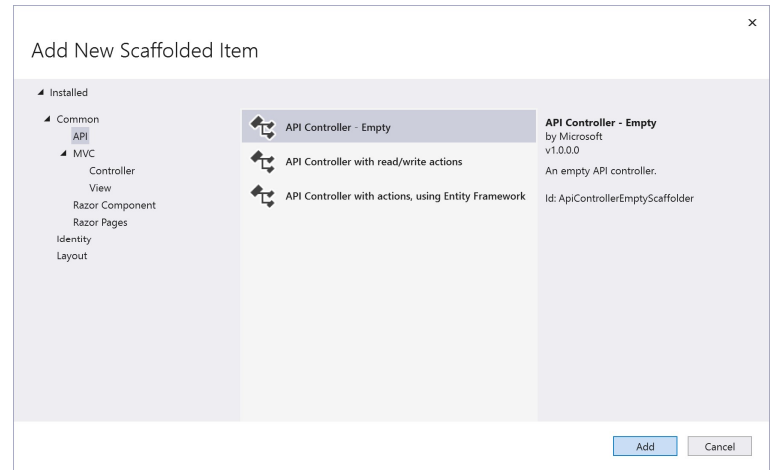
```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllers();

    services.AddDbContext<MovieDbContext>(options =>
        options.UseInMemoryDatabase(databaseName: "MovieDB"));

    services.AddSwaggerGen(c =>
    {
        c.SwaggerDoc(name: "v1", info: new OpenApiInfo { Title = "MovieService", Version = "v1" });
    });
}
```

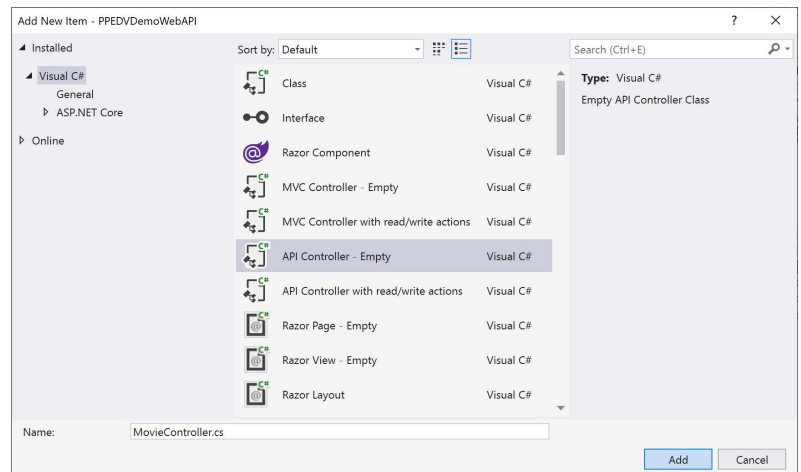
Erster RESTful-Service – Controller erstellen (Part1)

- 1.) Rechtsklick auf das Verzeichnis Controllers
- 2.) Wählen Sie API-Controller-Empty aus



Erster RESTful-Service – Controller erstellen (Part2)

- 1.) API-Controller nennen wir „MovieController.cs“
- 2.) Danach klicken wir auf „Add“
- 3.) Im Verzeichnis Controller, wird der neu erstellte Controller angezeigt



Erster RESTful-Service – Controller Implementierung

1.) API-Controller nennen wir

„MovieController.cs“

2.) Danach klicken wir auf „Add“

3.) Im Verzeichnis Controller, wird der neu

erstellte Controller angezeigt

```
namespace MovieService.Controllers
{
    [Route(template: "api/[controller]")]
    [ApiController]
    0 references
    public class MovieController : ControllerBase
    {
    }
}
```

Erster RESTful-Service – MovieDbContext in MovieController verfügbar machen

```
namespace MovieService.Controllers
{
    [Route(template: "api/[controller]")]
    [ApiController]
    1 reference
    public class MovieController : ControllerBase
    {
        private MovieDbContext dbContext;

        0 references
        public MovieController(MovieDbContext movieDbContext)
        {
            dbContext = movieDbContext;
        }
    }
}
```

Erster RESTful-Service – Implementieren von HTTP-Methoden

```
[HttpGet]
0 references
public IList<Movie> GetAllMovies() => dbContext.Movies.ToList();

[HttpPost]
0 references
public void AddMovie(Movie movie)
{
    dbContext.Add(movie);
    dbContext.SaveChanges();
}

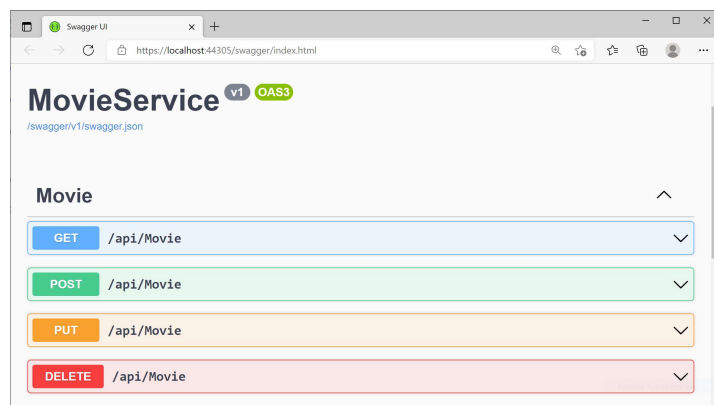
[HttpPut]
0 references
public void Update(Movie movie)
{
    dbContext.Update(movie);
    dbContext.SaveChanges();
}

[HttpDelete]
0 references
public void Delete (Movie movie)
{
    dbContext.Remove(movie);
    dbContext.SaveChanges();
}
```

27

Erster RESTful-Service – Projekt ausführen

- 1) Das Projekt können wir über die Menüleiste mit folgenden Icon starten:
Alternativ können wir auch F5 verwenden.
- 2) In Swagger sehen wir unsere erstellen Methoden und können diese dort auch ausführen



Return Values

Primitives, IEnumerables, ActionResult

Return Values – native Datentypen

- Die einfachste Aktion gibt einen primitiven Datentyp zurück (z.B. string oder einen benutzerdefinierten Objekttyp)

```
[Route(template: "api/[controller]")]
[ApiController]
0 Verweise
public class ReturnValuesController : ControllerBase
{
    [HttpGet(template: "StringSample")]
    0 Verweise
    public string GetVersion()
    {
        return "Willkommen bei ppedv!";
    }

    [HttpGet(template: "BoolSample")]
    0 Verweise
    public bool GetBoolean()
    {
        return true;
    }

    [HttpGet(template: "IntegerSample")]
    0 Verweise
    public int GetInteger()
    {
        return 123;
    }

    [HttpGet(template: "DateTimeSample")]
    0 Verweise
    public DateTime TheTime()
    {
        return DateTime.Now;
    }
}
```

30

Return Values – spezifischer Datentypen

- Benutzdefinierte Datentypen würden in diesem Fall ausreichen und vom Sicherheits Aspekt:
 - Da keine Methoden-Parameter verwendet werden, ist auch keine Validierung der Parameter nötig.

```
[HttpGet(template: "ComplexTypeSample")]
0 Verweise
public Product GetMockProduct()
{
    return new Product()
    {
        Id = 1,
        Name = "Xamarin Kurs",
        Description = "Einsteigerkurs",
        IsOnSale = true
    };
}

[HttpGet]
0 Verweise
public List<Product> Get() =>
    _repository.GetProducts();
```

31

Return Values – IEnumerable<T>

- Serialisierungsprogramm wird bei Rückgabe mit `yield return` verwendet.
- Ab ASP.NET Core 3.0 gilt bei `IEnumerable` und `IAsyncEnumerable`:
 - Es kommt nicht mehr zu synchroner Iteration.
 - Gleiche Effizienz wie bei Rückgabe von `IEnumerable<T>`.
- Ab ASP.NET Core 3.0 puffert das Ergebnis der folgenden Aktion, bevor es dem Serialisierungsprogramm bereitgestellt wird

```
[HttpGet(template: "syncsale")]  
0 references | 0 changes | 0 authors, 0 changes  
public IEnumerable<Product> GetOnSaleProducts()  
{  
    var products = _repository.GetProducts();  
  
    foreach (var product in products)  
    {  
        if (product.IsOnSale)  
        {  
            yield return product;  
        }  
    }  
}
```

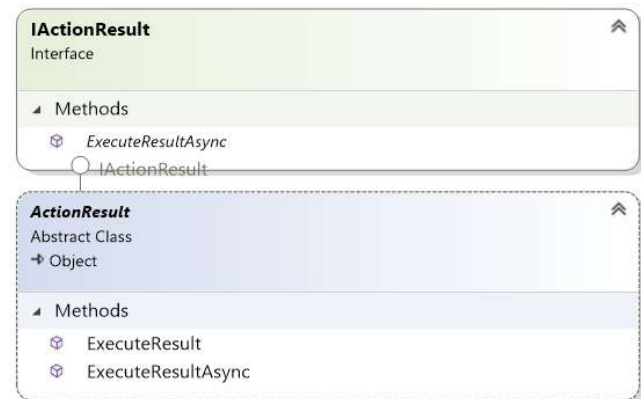

Return Values – IEnumerable<T>

- Microsoft empfiehlt zwischen IEnumerable<T> oder IEnumerableAsync<T> den asynchronen Ansatz, um die asynchrone Iteration sicherzustellen.
- Letztendlich basiert der Iterationsmodus auf dem zugrunde liegenden Rückgabetyt.
- MVC puffert automatisch jeden konkreten Typ, der IEnumerable<T> implementiert.

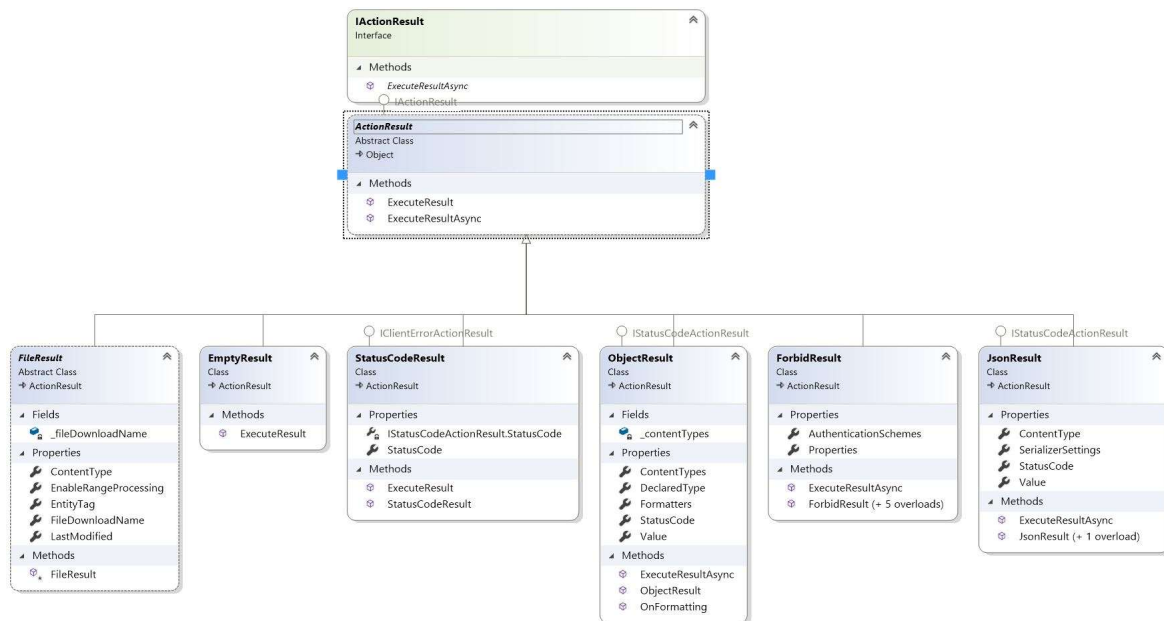
```
[HttpGet(template: "asynsale")]  
0 references | 0 changes | 0 authors, 0 changes  
public async IEnumerable<Product> GetOnSaleProductsAsync()  
{  
    var products = _repository.GetProductsAsync();  
  
    await foreach (var product in products)  
    {  
        if (product.IsOnSale)  
        {  
            yield return product;  
        }  
    }  
}
```

Return Values – IActionResult / ActionResult

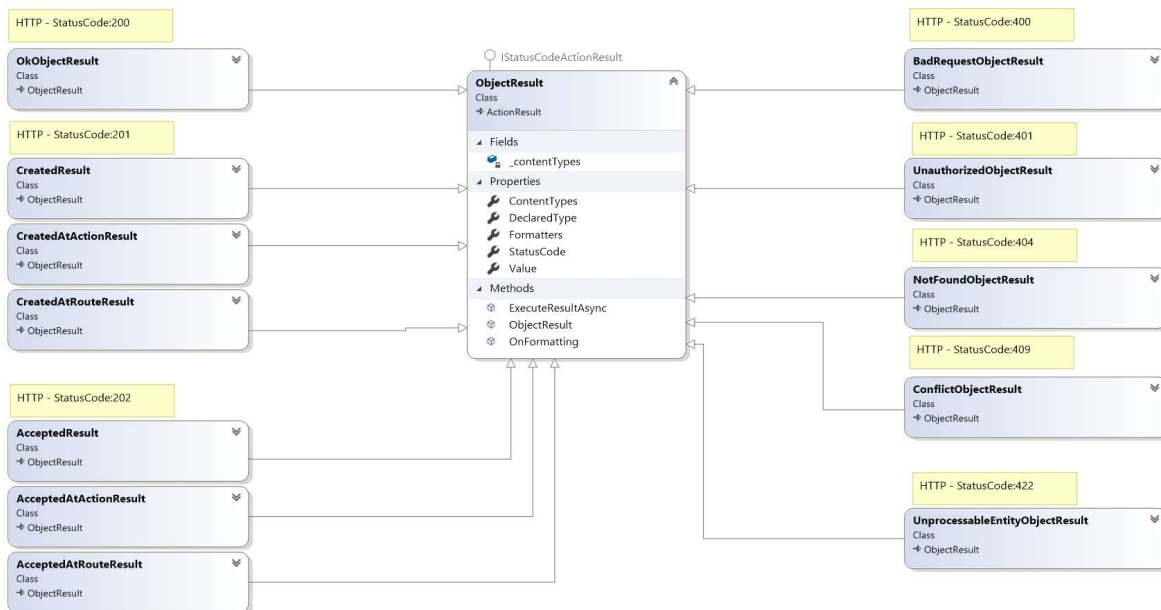
- Neben den vorherigen .NET Datentypen ist IActionResult die höchste Abstraktion, was bezüglich der Rückgabe-Datentypen einer Controller-Klasse betrifft.
- ActionResult ist eine abstrakte Klasse und wird von vielen Klassen abgeleitet, die wiederum als Basisklassen für spezielle Aufgaben dient. (siehe nächste Folie)



Return Values – ActionResult - Familienstammbaum



Return Values – Abgeleitete Klassen von ObjectResult

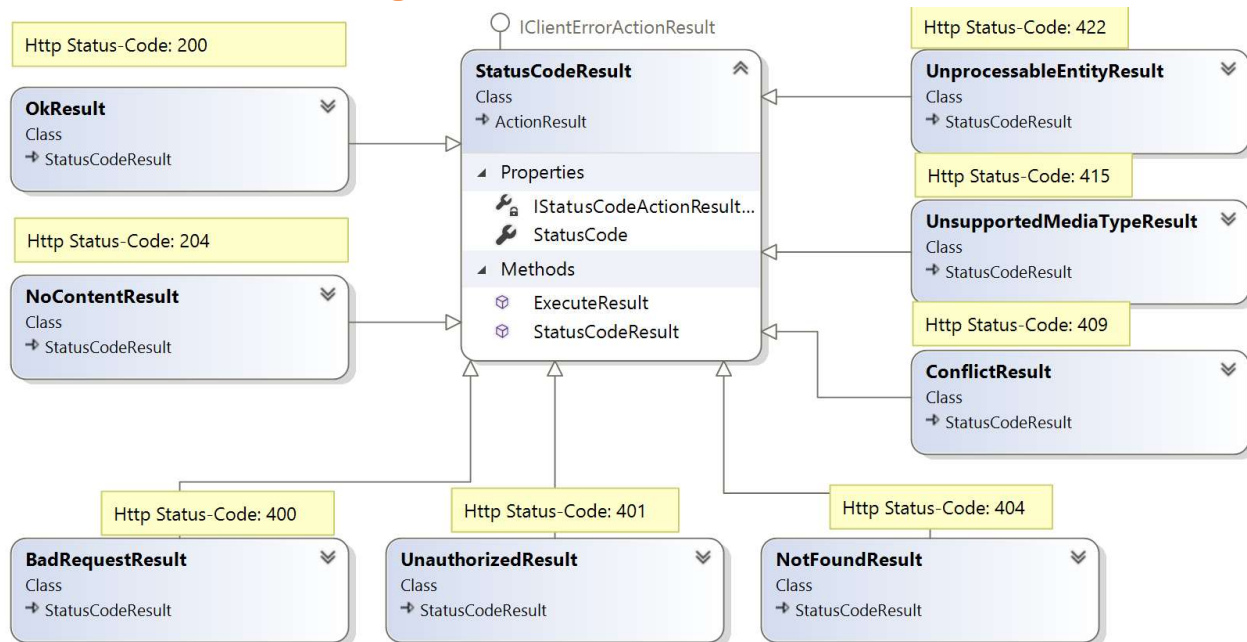


Return Values – Abgeleitete Klassen von ObjectResult im Einsatz

Alle abgeleitete Klassen von `ObjectResult` sind in `ControllerBase` als Methoden-Rückgabebetypen in folgenden Methoden implementiert.

ControllerBase	
Abstract Class	
↳ Object	
▸ Fields	
▸ Properties	
▾ Methods	
⊞ Accepted (+ 5 overloads)	
⊞ AcceptedAtAction (+ 5 overloads)	
⊞ AcceptedAtRoute (+ 4 overloads)	
⊞ BadRequest (+ 2 overloads)	
⊞ Conflict (+ 2 overloads)	
⊞ Created (+ 1 overload)	
⊞ CreatedAtAction (+ 2 overloads)	
⊞ CreatedAtRoute (+ 2 overloads)	
⊞ NotFound (+ 1 overload)	
⊞ Ok (+ 1 overload)	
⊞ Unauthorized (+ 1 overload)	
⊞ UnprocessableEntity (+ 2 overloads)	

Return Values – Abgeleitete Klassen von StatusCodeResult

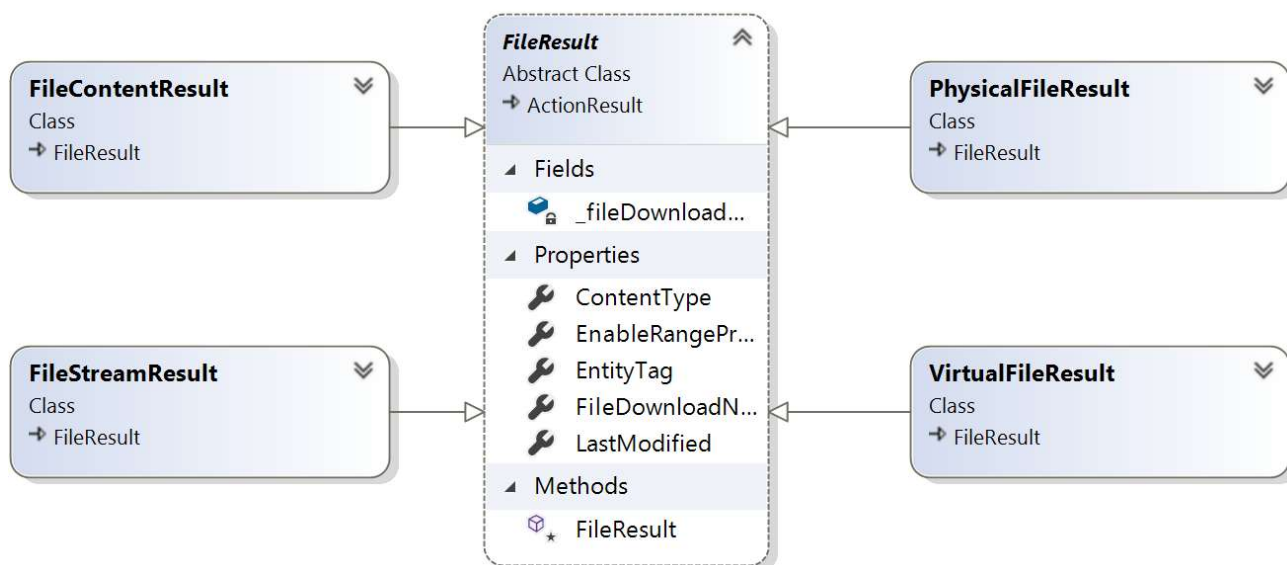


Return Values – Abgeleitete Klassen von StatusCodeResult

Alle abgeleitete Klassen von `StatusCodeResult` sind in `ControllerBase` als Methoden-Rückgabetypen in folgenden Methoden implementiert:



Return Values – Abgeleitete Klassen von FileResult



Alle von FileResult abgeleiteten Klassen werden in der ControllerBase->File-Methode als Rückgabetypen angeboten.

Return Values – IActionResult – Synchrones Beispiel

- Bei der folgenden synchronen Aktion gibt es zwei mögliche Rückgabetypen.
- Die NotFound-Hilfsmethode (Rückgabe von 404-Statuscode) wird als Kurzform für `return new NotFoundResult();` aufgerufen.
- Ein 200-Statuscode wird mit dem Product-Objekt zurückgegeben, wenn das Produkt vorhanden ist.
- Die Ok-Hilfsmethode wird als Kurzform für `return new OkObjectResult(product);` aufgerufen.

```
[HttpGet(template: "{id}")]
1 reference | Scott Addie, 200 days ago | 1 author, 1 change | 1 work item
public IActionResult GetById(int id)
{
    if (!_repository.TryGetProduct(id, out var product))
    {
        return NotFound();
    }

    return Ok(product);
}
```

Return Values – IActionResult – Asynchrones Beispiel

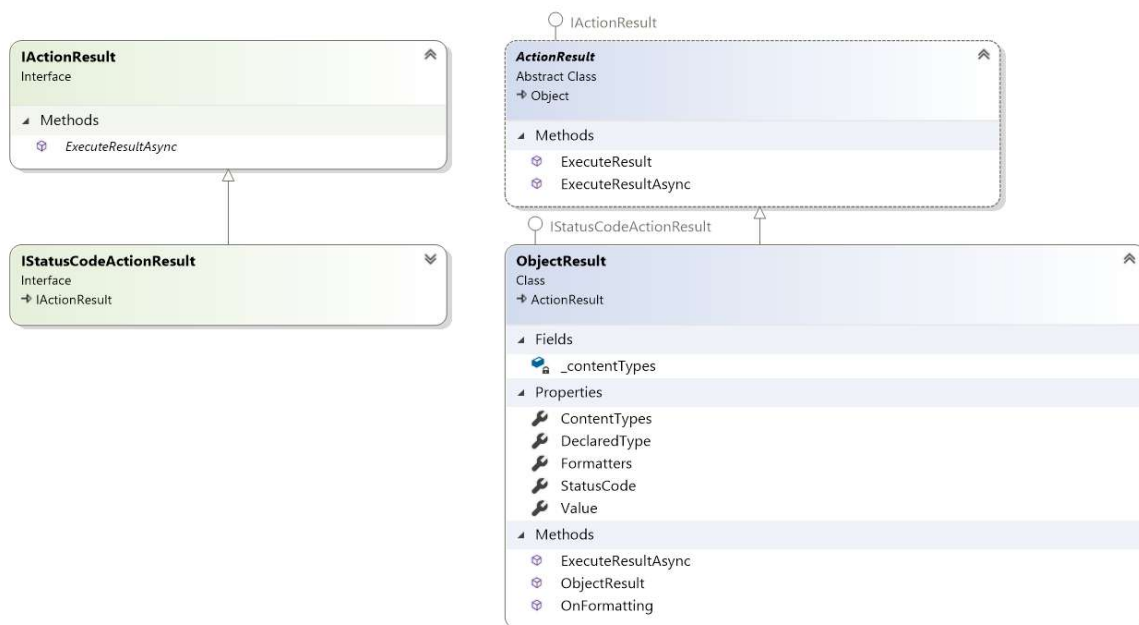
- Ein 400-Statuscode wird zurückgegeben, wenn die Produktbeschreibung „XYZ-Widget“ enthält.
- Die BadRequest-Hilfsmethode wird als Kurzform für `return new BadRequestResult();` aufgerufen.
- Ein 201-Statuscode wird von der Hilfsmethode `CreatedAtAction` generiert, wenn ein Produkt erstellt wird.
- Eine Alternative zum Aufrufen von `CreatedAtAction` ist: `return new CreatedAtActionResult(nameof(GetById), "Products", new { id = product.Id }, product);`

```
[HttpPost]
0 references | 0 changes | 0 authors, 0 changes
public async Task<IActionResult> CreateAsync(Product product)
{
    if (product.Description.Contains(value: "XYZ Widget"))
    {
        return BadRequest();
    }

    await _repository.AddProductAsync(product);

    return CreatedAtAction(nameof(GetById), new { id = product.Id }, product);
}
```

Return Values – IActionResult / ObjectResult Architektur



Return Values – ActionResult<T>

- ASP.NET Core 2.1 wurde der Rückgabotyp ActionResult <T> für Web-API-Controlleraktionen eingeführt.
- Damit wird die Rückgabe eines von ActionResult abgeleiteten Typs oder eines spezifischen Typs ermöglicht. (siehe nächste Folie)
- ActionResult<T> besitzt gegenüber dem IActionResult-Typ die folgenden Vorteile:
 - [ProducesResponseType(200, Type = typeof(Product))] wird zu [ProducesResponseType(200)] vereinfacht.
 - Der erwartete Rückgabotyp der Aktion wird stattdessen von T in ActionResult<T> abgeleitet.
 - Implizite Umwandlungsoperatoren unterstützen die Konvertierung von T und ActionResult in ActionResult<T>.
 - T wird in ObjectResult konvertiert, wodurch return new ObjectResult(T); zu return T; vereinfacht wird.

Return Values – ActionResult<T> - View in Sourcecode 1/2

1.) Definition von ActionResult<Tvalue>:

```
public sealed class ActionResult<TValue> : IConvertToActionResult
```

2.) Definition von IConvertToActionResult

```
public interface IConvertToActionResult  
{  
    2 references | 0 changes | 0 authors, 0 changes  
    IActionResult Convert();  
}
```

Return Values – ActionResult<T> - View in Sourcecode 2/2

3.) Implementierung des IConvertToActionResult in ActionResult<Tvalue>

```
IActionResult IConvertToActionResult.Convert()
{
    if (Result != null)
    {
        return Result;
    }

    int statusCode;
    if (Value is ProblemDetails problemDetails && problemDetails.Status != null)
    {
        statusCode = problemDetails.Status.Value;
    }
    else
    {
        statusCode = DefaultStatusCode;
    }

    return new ObjectResult(Value)
    {
        DeclaredType = typeof(TValue),
        StatusCode = statusCode
    };
}
```

Return Values – ActionResult<T> – Synchroner Aktion

- Ein 404-Statuscode wird zurückgegeben, wenn das Produkt in der Datenbank nicht vorhanden ist.
- Ein 200-Statuscode wird mit dem entsprechenden Product-Objekt zurückgegeben, wenn das Produkt vorhanden ist.
- Vor ASP.NET Core 2.1 hätte die Zeile
`return product;`
stattdessen
`return Ok(product);`
lauten müssen.

```
[HttpGet(template: "{id}")]
1 reference | 0 changes | 0 authors, 0 changes
public ActionResult<Product> GetById(int id)
{
    if (!_repository.TryGetProduct(id, out var product))
    {
        return NotFound();
    }

    return product;
}
```

Konventionen

HTTP-Verbs, HTTP-Statuscodes

WebAPI Controller: Default – WebAPI Konventionen

HTTP Method	Possible Web API Action Method Name	Usage
GET	Get()	Retrieves data.
	GetAllStudent() // jeder Name der mit Get startet	
POST	Post()	Inserts new record.
	PostNewStudent() // jeder Name der mit Post startet	
PUT	Put()	Updates existing record.
	PutStudent() // jeder Name der mit Put startet	
PATCH	Patch()	Updates record partially.
	PatchStudent() // jeder Name der mit Patch startet	
DELETE	Delete()	Deletes record.
	DeleteStudent() // jeder Name der mit Delete startet	

WebAPI Controller – Default Übersicht

```
[ApiController]
[Route(template: "[controller]")]
1 reference
public class PeopleController : ControllerBase
{
    private readonly PeopleDbContext _context;
    0 references
    public PeopleController(PeopleDbContext context) {...}

    0 references
    public async Task<ActionResult<IEnumerable<Person>>> GetPerson() {...}
    0 references
    public ActionResult<Person> GetPerson(int id) {...}
    0 references
    public IActionResult PutPerson(int id, Person person) {...}
    0 references
    public IActionResult PostPerson(Person person) {...}
    0 references
    public IActionResult DeletePerson(int id) {...}
}
```

WebAPI Konventionen – HTTP-Verbs

```
[Route(template: "api/{controller}")]
[ApiController]
1 reference
public class PeopleController : ControllerBase
{
    private readonly PeopleDbContext _context;
    0 references
    public PeopleController(PeopleDbContext context) {...}

    [HttpGet]
    0 references
    public async Task<ActionResult<IEnumerable<Person>>> GetPerson() {...}

    [HttpGet(template: "{id}")]
    0 references
    public ActionResult<Person> GetPerson(int id) {...}

    [HttpPut(template: "{id}")]
    0 references
    public IActionResult PutPerson(int id, Person person) {...}

    [HttpPost]
    0 references
    public ActionResult<Person> PostPerson(Person person) {...}

    [HttpDelete(template: "{id}")]
    0 references
    public IActionResult DeletePerson(int id) {...}
}
```

- HTTP Verbs können kombiniert werden
- HTTP Verbs helfen Swagger bei der Interpretierung der WebAPI (Konventionen).
- Daten werden per Default als JSON übertragen
- Dependency Injection

Open API / SwaggerUI

• Open API / Swagger / SwaggerUI

- WebAPI Namenskonvention -> Dokumentation wird generiert
- WebAPI – Methoden können getestet werden
- Dokumentation ist Versionierbar
- Mithilfe von swagger.json kann man Client generieren (seit .NET 5)
- Open API wird von den meisten Cloud – Anbieter unterstützt



52

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

WebAPI Controller - HTTP-Status Codes - Übersicht

LEVEL 200 SUCCESS

200 – OK
201 – Created
204 – No Content

LEVEL 400 CLIENT ERROR

400 – Bad Request
401 – Unauthorized
403 – Forbidden
404 – Not Found
409 – Conflict

LEVEL 500 SERVER ERROR

500 – Internal
Server Error

WebAPI Controller - HTTP-Status Codes - Detailliert

Code	Nachricht	Bedeutung
200	OK	Erfolg, das Ergebnis der Anfrage wird in der Antwort übertragen.
201	Created	Erfolgreich bearbeitet, die Ressource wurde vor dem Senden der Antwort erstellt.
202	Accepted	Anfrage akzeptiert, wird aber erst später ausgeführt. Kein Gelingen garantiert
301	Moved Permanently	Redirect, die angeforderte Ressource steht unter einer anderen Adresse bereit. Die alte Adresse ist nicht länger gültig.
304	Not Modified	Mit Web Api Caching
400	Bad Request	Fehlerhafte Anfrage
401	Unauthorized	Keine gültige Authentifizierung, Anfrage nicht möglich.
402	Payment Required	Status für zukünftige HTTP-Protokolle reserviert – „Bezahlung benötigt“. Wird oft verwendet um Fehler im Validierungsprozess aufzudecken.
403	Forbidden	Client nicht berechtigt, Anfrage wurde nicht durchgeführt.
404	Not Found	Die angeforderte Ressource wurde nicht gefunden. Anfrage ohne Grund abgewiesen.
409	Conflict	Anfrage wurde unter falschen Annahmen gestellt.
500	Internal Server Error	Unerwarteter Serverfehler.